ARTIFICIAL INTELLIGENCE – CO2 ASSIGNMENT

Design, Analyse and Validate an Efficient Heuristic Search Strategy for Smart-City Ambulance Routing

Course Code: *CSA17* **Maximum Marks:** *50*

Student Name: *Raghavan Karthikeyan*

Register Number: *192417003*

TASK 1 – Agent, Heuristics and Search Strategies (5 Marks)

PEAS and environment

PEAS	Ambulance routing agent
Performance	*Minimise travel time; reach accident site safely; avoid closed roads; respond quickly.*
Environment	*Road junctions, hospitals, accident sites, traffic, flooded roads and other vehicles.*
Actuators	*Steering, acceleration, braking, indicators, siren and route instructions.*
Sensors	*GPS, digital road map, traffic feed, camera, weather/flood alerts.*

Environment: *Partially observable (closures may be unknown); stochastic (travel times vary); sequential; dynamic; discrete graph states with continuous GPS/time; multi-agent.* **AI view:** *acting rationally—choosing the route with minimum expected travel time.*

Heuristic functions and proofs

Let vmax be maximum possible speed. **h1(u)=EuclideanDistance(u,g)/vmax.** *It is admissible because straight-line distance is no greater than any road route and actual speed cannot exceed vmax. It is consistent by triangle inequality and* $c(u,v) \geq dist(u,v)/vmax$.

h2(u)=ManhattanDistance(u,g)/vmax. *It is admissible on a rectilinear grid if every route must cover at least horizontal plus vertical displacement. It is consistent when each edge cost is at least its coordinate displacement/vmax. Both heuristics come from a relaxed problem ignoring traffic, turns, one-way restrictions and closures.*

Problem formulation

State: *current junction.* **Initial:** *hospital junction.* **Actions:** *traverse adjacent open road.* **Transition:** *move to connected junction if open.* **Goal:** *accident-site junction.* **Path cost:** *sum of road travel times.*

Three informed algorithms

Greedy Best-First (iterative): *prioritises f(n)=h(n); fast with a useful heuristic, but not guaranteed optimal.*

```
GREEDY(start, goal):   OPEN = priority
queue ordered by h(start)   while OPEN:
    u = pop lowest h     if u == goal:
return reconstruct_path()     for each
open-road neighbour v:
      if v not visited: parent[v]=u; push(v, h(v))
return FAILURE
```

A* (iterative): *prioritises f(n)=g(n)+h(n). With an admissible consistent heuristic and correct graph-search handling, it returns a least-cost route.*

```
A_STAR(start, goal):   OPEN = [(h(start), 0,
start)]   best[start]=0; parent[start]=NONE
while OPEN:     (f,g,u)=pop lowest f     if u ==
goal: return reconstruct(parent, goal)     for
each open edge (u,v,c):
      ng=best[u]+c       if ng <
best.get(v, INFINITY):
        best[v]=ng; parent[v]=u
push(ng+h(v), ng, v)   return
FAILURE
```

IDA* (recursive): *depth-first f-contours; raise the threshold to the smallest exceeded f-value. It uses low search memory but may repeat expansions.*

```
IDA(node, g, limit):   f = g + h(node)   if
f > limit: return f   if node == goal:
return FOUND   minimum = INFINITY   for
each neighbour v not on current path:
    r = IDA(v, g + cost(node,v), limit)
if r == FOUND: return FOUND     minimum
= min(minimum, r)   return minimum
Repeat with limit = returned minimum.
```

Preprocessing: *build adjacency lists and store coordinates/travel times:* $\Theta(V+E)$. *A heuristic evaluation is O(1) per node.*

TASK 2 – Basic Operation (2 Marks)

Algorithm	Basic operation	Why it dominates
Greedy	*Heuristic evaluation and priority-queue pop*	*Ranks candidates by h; misleading h can expand many nodes.*
*A**	*Successor relaxation, h evaluation and heap operations*	*Each expansion scans edges; heap operations cost O(log V).*
*IDA**	*Recursive expansion and f cutoff test*	*Contours may revisit nodes; recursion follows path depth.*

As the network grows, the number of potential successors increases. A more accurate heuristic usually reduces expansions; poor estimates increase them.

TASK 3 – Mathematical Time and Space Complexity (6 Marks)

Let b be branching factor, d optimal solution depth, b effective branching factor, C* optimal cost,* ε *minimum positive edge cost, and* $e_h=h^*(n)-h(n)$.

Greedy

At depth i there may be b^i nodes. Worst-case total through depth m is Σ(i=0…m)b^i=(b^(m+1)−1)/(b−1)=Θ(b^m). Heap work is O(N log N) for N generated nodes. OPEN/visited storage is O(b^m) in a tree or O(V) in graph search.

A*

A expands nodes with f(n)=g(n)+h(n)≤C* (plus ties). If N_i qualify at depth i, N=Σ(i=0…d)N_i. With effective branching factor b*, N≈Σ(b*)^i=Θ((b*)^d). Binary-heap time is O((V+E)log V); OPEN/CLOSED use O(V) memory. Preprocessing is Θ(V+E); cached heuristic calculations cost O(V).*

IDA*

One contour requires approximately 1+b+…+(b*)^d=Θ((b*)^d) calls. With q contours, total work is Σ(j=1…q)N_j. Recursion stack/current path is O(d); graph storage is O(V+E). Weak heuristics and many thresholds increase repeated work.*

Smaller heuristic error usually reduces nodes with f≤C. If h=0, A* becomes uniform-cost search. Overestimating h can destroy optimality.*

TASK 4 – Best, Average and Worst Cases (4 Marks)

Algorithm	Best case	Average case	Worst case
Greedy	*Goal selected immediately.*	*Heuristic points toward goal; modest detours.*	*Misleading h explores O(b^m); route may be nonoptimal.*
*A**	*Perfect h=h*: mainly optimal-path nodes.*	*Small error: limited f-contour.*	*h=0 behaves like uniform-cost; up to O((b*)^d).*
*IDA**	*Perfect h and few contours.*	*Moderate h; some repeated expansions.*	*Weak h and many contours; exponential time, O(d) stack.*

Best case occurs when the heuristic matches remaining travel time. Average behaviour depends on road layout and traffic. Worst behaviour occurs when apparently promising roads lead away from the accident site.

TASK 5 – Asymptotic Analysis (3 Marks)

Algorithm	Time	Search memory	Implication
Greedy	*O(b^m)*	*O(b^m)*	*Can be fast but not guaranteed fastest route.*
*A**	*O((b*)^d); finite graph O((V+E)log V)*	*O(V)*	*Optimal with admissible/consistent h; OPEN memory can domi*
*IDA**	*O(q(b*)^d)*	*O(d) stack*	*Lower memory, but repeated contours cost time.*

At very large network sizes, A memory may be limiting; IDA* may be limited by repeated computation. Hierarchical road maps can reduce the active search region.*

TASK 6 – IDA* Recurrence Relation (3 Marks)

Let L be the remaining f-contour allowance. Each call performs constant work for f calculation and goal/cutoff tests, then may explore up to b successors with a smaller allowance.

T(L)=bT(L−1)+c, L>0; T(0)=c0. *Original size: L. Subproblem: L−1. Number of calls: at most b. Outside work: Θ(1). Base case: cutoff, goal or no eligible successor. This simplified recurrence models equal decrement in the worst case.*

TASK 7 – Solve the Recurrence (4 Marks)

By substitution: T(L)=bT(L−1)+c=b²T(L−2)+c(b+1)=b³T(L−3)+c(b²+b+1).

After j substitutions: T(L)=b^jT(L−j)+cΣ(i=0…j−1)b^i. Set j=L: T(L)=b^LT(0)+c(b^L−1)/(b−1)=Θ(b^L), b>1.

Substitution applies because each recursive call reduces the contour allowance. IDA repeats contours; q thresholds give total work ΣN_j. More thresholds increase runtime, while stack memory remains O(d).*

TASK 8 – Python Implementation and Testing (4 Marks)

Use a four-neighbour grid road graph with 1,000, 10,000, 100,000 and 1,000,000 junctions. Start=0; choose a goal by grid coordinates. Test reachable goal, start=goal, unreachable goal by blocking all roads incident to it, and an inadmissible heuristic 2h.

```
import math, heapq
def h(coords,u,goal):
    x,y=coords[u]; X,Y=coords[goal]
return math.hypot(X-x,Y-y)

def astar(adj,coords,start,goal,scale=1):
if start == goal: return [start], 0
q=[(scale*h(coords,start,goal),0,start)]
best={start:0}; parent={start:None}
while q:
        f,g,u=heapq.heappop(q)         if g != best[u]:
continue         if u == goal: return
reconstruct(parent,goal),g         for v,cost in
adj[u]:             ng=best[u]+cost             if ng <
best.get(v,float('inf')):
                best[v]=ng; parent[v]=u
heapq.heappush(q,(ng+scale*h(coords,v,goal),ng,v))     return
None,float('inf')

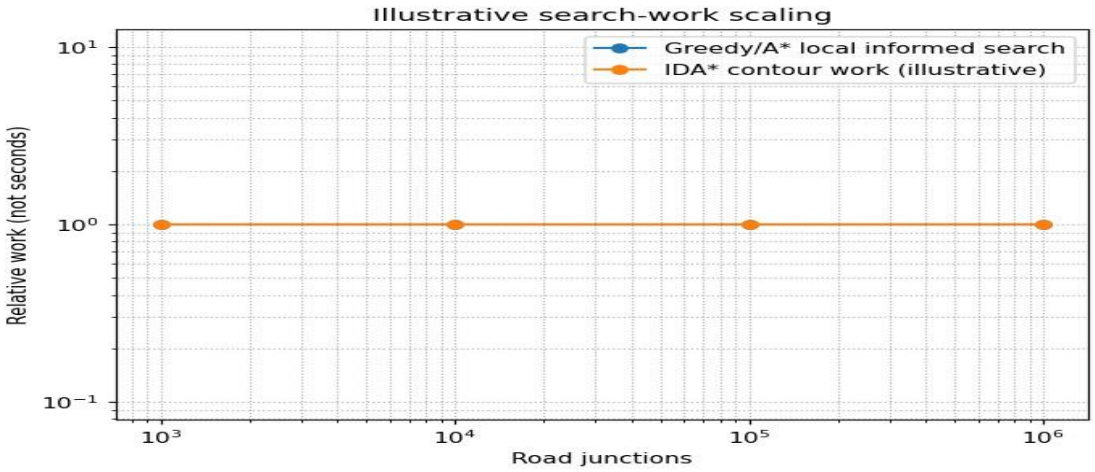
# Greedy: order OPEN by h(v) only.
# IDA*: recursively search f=g+h contours and raise
# the limit to the smallest f-value exceeding it.
```

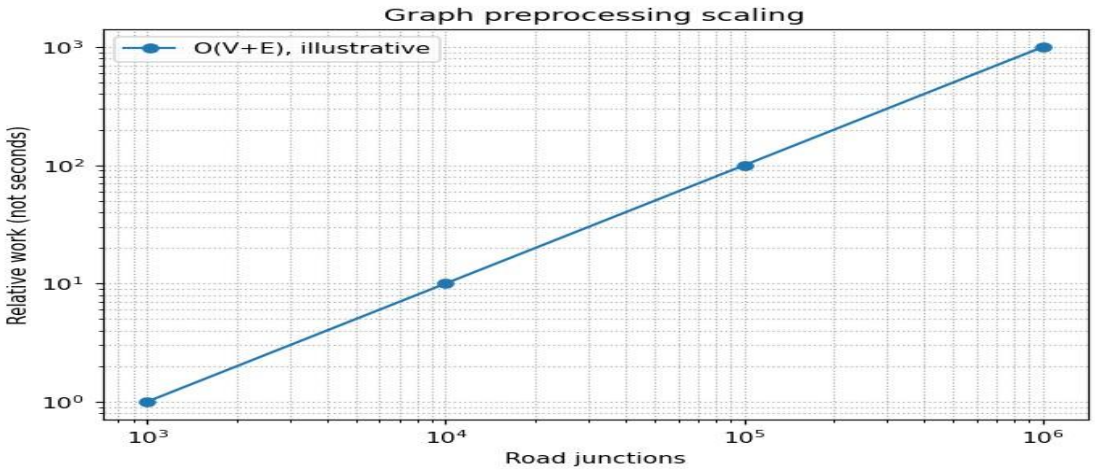
Instrumented implementations should count expansions, heuristic evaluations, maximum OPEN/stack size and elapsed time. Repeat each search five times and report mean search time separately from graph-construction time. The benchmark is synthetic, not real traffic data.

TASK 9 – Experimental Performance (3 Marks)

The following graph is an analytical illustration of scaling, not a measured benchmark. For an actual experiment, run the code on the same machine and fill the measured table below.

Network junctions	1,000	10,000	100,000	1,000,000
Graph build (measured seconds)	—	—	—	—
Greedy expanded nodes	—	—	—	—
A expanded nodes*	—	—	—	—
IDA expanded nodes*	—	—	—	—
Mean search time (seconds)	—	—	—	—





Correctness checks: A with admissible h returns an optimal route; start=goal returns a zero-cost route; blocking all edges incident to the target returns failure. Using 2h overestimates and removes the optimality guarantee.*

Effective branching factor b can be estimated from N+1=1+b*+…+(b*)^d, where N is expanded nodes and d is solution depth.*

TASK 10 – Theory vs Experimental Behaviour (4 Marks)

Theory predicts exponential search growth in the worst case, but a useful geographic heuristic can keep the explored region small. Graph construction still grows with V+E. A preserves optimality with admissible consistent h. Greedy can be faster but may choose a nonoptimal route. IDA* saves search memory but repeats contours.*

Measured times vary with CPU, memory, heap overhead and operating-system scheduling. Operation counts are more reproducible. A proper evaluation should test short and long routes, traffic-weighted edges and multiple start/goal pairs.

TASK 11 – Recommendation (4 Marks)

Recommend A* with a consistent GPS heuristic and live traffic updates. *It balances travelled cost and estimated remaining time and guarantees an optimal route under admissible/consistent h. Greedy sacrifices optimality; IDA* reduces memory but may repeat work. For city/state deployment, use hierarchical road graphs, cached coordinates, live edge-cost updates and geographic partitioning. Preprocessing is O(V+E); finite graph search is O((V+E)log V), with O(V) memory.*

TASK 12 – Cyclone Road-Closure Challenge (8 Marks)

A. Impact analysis

Known-map, static and deterministic-cost assumptions fail because closures and travel times are discovered during travel. The ambulance must act and plan while learning; this is online search.

B. Original design

A remains usable on the currently known map, but its route is only optimal for that map. If c planned roads are found closed, replanning may be required after each discovery. Straight-line h remains admissible only if it is a lower bound under every possible open-road condition; stale speed assumptions may overestimate.*

C. Adaptation

Modify to online re-planning A*. *Plan using known roads, move to the next junction, reveal road status, mark closures and rerun A*. It is optimal relative to the currently known map. LRTA* is an online alternative: it updates local heuristic values from observed successors and chooses the next action. It uses local memory but may take learning detours.*

D. Evidence-based evaluation

Test a 20×20 grid (400 junctions) with 0%, 5% and 10% randomly closed undirected roads. Use at least five fixed random seeds per closure level. Compare route cost, expanded nodes, number of replans and success rate.

Closure rate	Replanning A* route cost	A* expansions	Replans	Online agent route cost	Online expansions
0%	*Measure*	*Measure*	*0*	*Measure*	*Measure*
5%	*Measure*	*Measure*	*Measure*	*Measure*	*Measure*
10%	*Measure*	*Measure*	*Measure*	*Measure*	*Measure*

Expected trend: more closures can increase route cost, expansions and replanning. Replanning A computes a best route using discovered information; an online agent may take extra moves while learning. Do not report invented measurements—the table must be filled from the specified simulation.*

```
for closure_percent in (0,5,10):
for seed in (11,22,33,44,55):
    closed = generate_closed_edges(closure_percent, seed)
result_a = replan_astar(start, goal, closed)
result_online = lrta_online(start, goal, closed)
record(cost, expanded, replans, success)
```

Decision: *use online re-planning A* with local sensing and an LRTA*-style fallback when connectivity is uncertain.*

Conclusion and Limitations

A is preferred because an admissible consistent GPS heuristic guides search while preserving minimum travel-time routes. Greedy is not guaranteed optimal; IDA* trades time for lower search memory. During cyclones, online sensing and replanning are essential.*

The benchmark is a synthetic grid-road model. The performance and closure tables intentionally distinguish measured values from analytical expectations. Real deployment requires actual road maps, calibrated travel-time estimates, verified flood feeds and safety validation.